Writing Test – M.M Rohan Nawarathne

01

Aspect	Functional Testing	Non-Functional Testing
Purpose	Verifies that the system behaves according to the specified functional requirements	Checks non-functional aspects like performance, usability, reliability, etc.
Focus	What the system does (features & functions)	How the system performs under various conditions
Examples	Login functionality, user registration, form submission, business logic	Load testing, stress testing, scalability, security, usability
Validation	Based on user requirements or use cases	Based on performance benchmarks or standards
Tools	Selenium, JUnit, TestNG, Postman	JMeter, LoadRunner, Lighthouse, New Relic
Test Basis	Functional specifications, use cases, business requirements	Quality attributes (performance, reliability, etc.), SLAs
Automation	Can be easily automated	Automation may be complex and require specialized tools
Type of Testing	Often black-box testing	Can involve black-box or white-box testing

Summary:

- **Functional Testing** = "Does it work as expected?"
- **Non-Functional Testing** = "How well does it work under different conditions?"

02

The **Software Testing Life Cycle (STLC)** is a systematic process followed during software testing to ensure high quality and defect-free software. It includes several well-defined phases, each with its own goals and deliverables

1. Requirement Analysis

- Purpose: Understand what needs to be tested.
- Activities:
  - Analyze requirements from a testing perspective.
  - Identify testable requirements.
  - Involve stakeholders to clarify doubts.
- Deliverables: Requirement Traceability Matrix (RTM), Automation feasibility report (if needed)

2. Test Planning

- Purpose: Define the strategy and scope for testing.
- Activities:
  - Create a test plan.
  - Estimate resources, time, and effort.
  - Identify tools and training needs.
- Deliverables: Test Plan document, Effort estimation

3. Test Case Design / Development

- Purpose: Write test cases to verify each function.
- Activities:
  - Create test cases and test scripts.
  - Review and get them approved.
  - Prepare test data.
- Deliverables: Test cases, test scripts, test data

4. Test Environment Setup

- Purpose: Prepare hardware and software to execute tests.
- Activities:
  - Set up the testing environment (servers, databases, tools).
  - Confirm with a smoke test (basic functionality check).
- Deliverables: Environment ready confirmation

5. Test Execution

- Purpose: Run the tests and log defects.
- Activities:
  - Execute test cases.
  - Log defects for failed tests.
  - Retest and perform regression testing.
- Deliverables: Test execution reports, Defect logs

6. Test Cycle Closure

- Purpose: Evaluate the testing process and results.
- Activities:
  - Prepare test summary report.
  - Analyze metrics (pass/fail rate, defect density, etc.).
  - Identify lessons learned and best practices.
- Deliverables: Test Summary Report, Final Test Metrics, Lessons learned

03.

1. Unit Testing

- Focus: Testing individual components or functions of the code.
- Who Performs It: Developers
- Purpose: Ensure that each unit of code works as intended.
- Example: Testing a login function that takes a username and password and returns true/false.
- Tools: JUnit, NUnit, PyTest

2. Integration Testing

- Focus: Testing the interaction between integrated units or modules.
- Who Performs It: Developers or Testers
- Purpose: Detect interface issues or data flow problems between modules.
- Example: Testing the interaction between the login module and the user profile module.
- Tools: Postman (for APIs), JUnit, TestNG

3. System Testing

- Focus: Testing the complete, integrated application as a whole.
- Who Performs It: QA/Testers
- Purpose: Validate that the system meets the specified requirements.
- Example: Testing the entire e-commerce site including product search, add to cart, and checkout.
- Tools: Selenium, QTP, TestComplete

4. Acceptance Testing

- Focus: Testing from the end-user or client's perspective.
- Who Performs It: End-users, clients, or QA team
- Purpose: Confirm the system is ready for production use.
- Example: A client tests the app to see if the online payment process works correctly.
- Types:
    - Alpha Testing – In-house by internal staff
    - Beta Testing – Released to a limited group of real users
- Tools: UAT test scripts, manual testing

04.

Boundary Value Analysis (BVA) and Equivalence Partitioning (EP) are both black-box testing techniques used to design effective test cases by minimizing redundancy and maximizing coverage.

1. Boundary Value Analysis (BVA)

- Definition: BVA focuses on testing the boundaries or edge conditions of input values. The idea is that errors often occur at the edges of input ranges.
- Technique: Test the minimum, maximum, and just outside the boundary values.

2. Equivalence Partitioning (EP)

- Definition: EP divides input data into valid and invalid partitions or classes where test cases are designed to cover each partition at least once.
- Goal: Reduce total number of test cases while still maintaining coverage.

05

A **test plan** is one of the most important documents in software testing. It serves as the **blueprint** for the entire testing process and ensures that testing is **well-organized, systematic, and traceable**.

1. Provides Clear Direction

- Defines what will be tested, how, when, and by whom.
- Helps the entire QA team follow a common approach and avoid confusion.

2. Ensures Complete Test Coverage

- Lists all the features to be tested and not tested, helping avoid gaps.
- Aligns testing with business and technical requirements.

3. Resource & Time Management

- Specifies resource requirements, such as tools, environments, and team members.
- Includes a test schedule with milestones, helping manage time effectively.

4. Risk Management

- Identifies potential risks and defines mitigation strategies.
- Helps stakeholders be proactive instead of reactive.

5. Improves Communication

- Acts as a reference for stakeholders, developers, testers, and managers.
- Keeps everyone informed about the testing scope, strategy, and status.

6. Helps in Measuring Progress

- Test plans include entry and exit criteria, and metrics to evaluate testing progress and quality.
- Enables tracking against goals like % of test cases executed, passed, or failed.

7. Facilitates Review & Audit

- A well-documented test plan is useful for internal reviews and external audits.
- Shows the project followed a structured and professional testing approach.

Key Components of a Test Plan (Brief Overview):

- Test objectives
- Scope (in-scope / out-of-scope features)
- Test strategy
- Resources and responsibilities
- Environment and tools
- Test deliverables
- Schedule
- Risks and assumptions

06.

Agile Testing vs. Traditional Testing

Aspect	Agile Testing	Traditional Testing (Waterfall)
Approach	Iterative and continuous	Sequential and phase-based
Development Model	Follows Agile (Scrum, Kanban)	Follows Waterfall or V-model
Testing Phase	Testing happens concurrently with development	Testing starts after development is complete
Tester Involvement	Testers are involved from day one	Testers are involved after coding is done
Requirement Changes	Flexible and adaptive to changing requirements	Difficult to accommodate changes once defined
Test Planning	Continuous planning during sprints	One-time planning at the beginning
Collaboration	Close collaboration between testers, developers, and product owners	Limited collaboration; mostly handled by separate teams
Feedback Cycle	Fast, with daily stand-ups and sprint reviews	Slow, feedback mostly after development is done
Documentation	Lightweight and just enough	Heavy and detailed documentation
Testing Types Focused	Emphasis on automation, exploratory, continuous testing	Emphasis on formal, structured testing (e.g., system, acceptance)

07.

A test case is a set of conditions or steps used to verify whether a feature or functionality of an application is working as expected.

Each test case includes:

- Input data
- Execution steps
- Expected result
- Actual result (after testing)

How to Write an Effective Test Case

1. Test Case ID

- Unique identifier, e.g., `TC_001_Login_Valid`

2. Test Case Title / Description

- Brief and clear name that describes what is being tested.
  Example: “Verify login with valid credentials”

3. Pre-conditions

- Any setup needed before running the test.
  Example: User must be registered with a valid username/password

4. Test Steps

- Step-by-step actions to perform the test.
  Example:
  1. Open login page
  2. Enter valid username and password
  3. Click on "Login" button

5. Test Data

- Input values used in the test.
  Example:

Username: `user@example.com`
Password: `123456`

6. Expected Result

- What should happen if the feature works correctly.
  Example: User is redirected to the dashboard.

7. Actual Result

- What actually happened during execution (filled during testing).

8. Status (Pass/Fail)

- Final outcome of the test (based on expected vs actual result).

08.

Types of Test Data

Type	Description	Example
Valid Data	Meets the expected input criteria	Correct username/password
Invalid Data	Incorrect or out-of-range inputs	Wrong email format, negative numbers
Boundary Data	At the edge of valid input ranges	Age = 18 (min), 60 (max)
Empty/Null Data	Fields left blank or NULL values	Empty "email" field
Large Data Sets	Stress or load testing inputs	Uploading a file with 10,000 records
SQL Injection / Malicious Data	To test for security vulnerabilities	`' OR '1'='1`

Significance of Test Data

1. Ensures Accurate Testing

- Without appropriate test data, test cases may not reflect real-world scenarios and bugs can go undetected.

2. Helps Discover Edge Cases

- Using boundary or invalid data can help identify system vulnerabilities or unexpected behaviors.

3. Essential for Automation

- Automated tests require well-prepared test data to run repeatably and reliably.

4. Improves Security Testing

- Using malicious or incorrect data helps test how well the system handles exceptions and potential attacks.

5. Supports Performance and Load Testing

- Large volumes of data simulate real user traffic, helping assess system scalability and performance under stress.

09.

Defect Severity vs. Defect Priority

Aspect	Defect Severity	Defect Priority
Definition	How serious the defect is in terms of system functionality	How urgently the defect needs to be fixed
Set By	Usually the tester or QA team	Usually the project manager or client
Focus	Technical impact	Business impact
Fix Order	Based on the effect on application	Based on urgency in release cycle

10.

1. Understand the Application

- Study requirements, user stories, or business goals
- Identify key features, workflows, and risk areas

2. Define a Test Charter (Mission)

- A test charter outlines what you want to test, how, and why.

3. Set Time Box

- Allocate a time limit for the session (usually 60–90 minutes)
- Known as a Session-Based Test Management (SBTM) technique

4. Start Exploring the App

- Use the application just like an end-user would
- Try different data combinations, workflows, edge cases
- Focus on areas like:
    - Error messages
    - Form validations
    - Broken links
    - UI/UX inconsistencies

5. Document Your Findings

- Take notes of:
    - Test paths explored
    - Bugs found
    - Screenshots/videos (if needed)
    - Questions or uncertainties

6. Log Defects

- Report bugs clearly with steps to reproduce
- Attach relevant logs or screenshots

7. Review and Debrief

- Discuss findings with the team
- Decide whether further exploration is needed

11.

Importance of Smoke Testing:

- Build stability: Confirms that the build is stable enough for further testing.
- Saves time: Prevents testers from spending time on detailed testing if the build is fundamentally broken.
- Quick feedback: Provides immediate feedback to developers about critical issues.

Importance of Sanity Testing.

- Checks for defects in specific areas after a new change or bug fix.
- Helps in early detection of issues in newly developed functionalities.
- Saves time: Ensures that changes haven't broken the system without going into exhaustive testing.
- Quick and efficient: Focuses only on the changed or impacted areas, reducing testing time.

12.

1. Ensures Stability After Changes

2. Helps Identify Unintended Side Effect

3. Facilitates Continuous Delivery

4. Validates Bug Fixes and Enhancements

5. Reduces Risk of Introducing New Defects

6. Improves Confidence in Software Updates

7. Ensures Compatibility with Other Systems

13.

What is UAT?

User Acceptance Testing (UAT) is a type of testing performed to ensure that the software meets business requirements and works as expected in real-world scenarios. The main goal of UAT is to verify that the application solves the problem it was built for, is easy to use, and fits into the user's workflow.

Who Performs UAT?

- End Users: Typically, actual users who will be using the application in a real environment.
- Clients/Stakeholders: In some cases, UAT may be performed by the client or business stakeholders who have defined the business requirements.
- Product Managers: They might also take part to ensure that the software aligns with the business goals and objectives.

14.

1. Load Testing

- Load Testing is the process of testing a system to determine its behavior under normal and expected peak load conditions.
- The goal is to identify how the system handles the expected volume of users or transactions over a defined period.

2. Stress Testing

- Stress Testing is the process of testing a system under extreme load conditions, far beyond what the system is designed to handle.
- The goal is to find the breaking point or the maximum limit of the system and assess how it recovers from failure.

3. Performance Testing

- Performance Testing is a broad term that encompasses multiple tests designed to measure how well the system performs under various conditions.
- It focuses on evaluating the speed, responsiveness, and stability of the application under different workloads.

15.

What is API Testing?

API testing involves sending requests to an API, verifying that the responses are correct, and checking that the API performs as expected under various conditions. This testing focuses on the business logic layer of the application, making it different from testing the user interface (UI).

Why is API Testing Important?

1. API is the Backbone of Modern Applications

- APIs are integral in enabling interactions between different systems, services, and databases. In modern applications, APIs facilitate communication between front-end and back-end, external services, databases, and third-party integrations.

2. Ensures Business Logic Works Correctly

- APIs often contain the core business logic of an application. By testing APIs directly, you can ensure that critical functionalities (like data processing, authentication, etc.) are working correctly, even before front-end components are built or integrated.

3. Faster and More Efficient

- API testing focuses on the back-end functionality, which can often be tested independently of the user interface (UI). This makes it faster to test and easier to identify and fix issues early in the development cycle.

4. Supports Automation

- APIs are highly suitable for automation because they don't require a graphical interface. Automated tests can be written for APIs and run continuously as part of a CI/CD pipeline, ensuring quick feedback.

5. Ensures Data Integrity

- APIs handle a lot of data exchanges between different parts of an application. Testing ensures that the data is correctly transferred between services, is in the right format, and maintains its integrity during the exchange.

6. Performance and Load Validation

- APIs often need to handle a high volume of requests. API testing helps identify performance bottlenecks and ensures the system can scale effectively to handle traffic spikes.

7. Security Verification

- APIs are often exposed to the internet, which makes them vulnerable to attacks. Security testing is crucial to verify that the API has proper authentication, encryption, and authorization mechanisms in place.

16.

1. Requirements Identification:
    - Step 1: List all the requirements from the project documentation (e.g., functional requirements, user stories, business requirements).
    - These could be functional, non-functional, or technical requirements.
2. Mapping Requirements to Test Cases:

    - Step 2: For each requirement, create corresponding test cases that validate whether the requirement is met.
    - Each requirement will be mapped to one or more test cases.
    - The RTM should include references to the specific test case IDs or test scenarios that verify each requirement.
3. Track Test Execution:
    - Step 3: As testing progresses, the RTM will track which test cases have been executed and their outcomes (Pass, Fail, or Not Executed).
    - This provides visibility into the coverage of testing and whether all requirements have been validated.
4. Identify Gaps:
    - Step 4: if a requirement is not mapped to a test case or if a test case is failing, the RTM helps identify gaps in test coverage or the need for additional testing.
5. Validation and Sign-Off:
    - Step 5: After the testing phase, the RTM can be used to show stakeholders and clients that all requirements have been tested, and it provides a traceable audit trail for sign-off.

17.

Stages of the Defect Lifecycle

1. New (Defect Identified)
2. Assigned
3. Open
4. Fixed
5. Retesting
6. Closed
7. Reopened

18.

1. Testing Shows the Presence of Defects

2. Exhaustive Testing is Impossible

3. Early Testing

4. Defects Clustering

5. Testing is Context-Dependent

6. Absence of Errors Fallacy

7. Continuous Improvement

8. Independent Testing

19.

1. Reduces Repetitive Tasks

2. Increases Test Coverage

3. Faster Feedback

4. Improved Consistency and Accuracy

5. Efficient Use of Resources

6. Enables Parallel Testing

7. Facilitates Continuous Integration and Continuous Testing

8. Helps in Regression Testing

9. Enables Faster Time-to-Market

10. Enhances Test Maintenance

20.

1. Time-Consuming

2. Human Error

3. Limited Test Coverage

4. Inability to Reproduce Tests Consistently

5. Difficulties in Managing Large-Scale Projects

6. Repetitive Nature of Testing

7. Limited Resources

8. Difficult to Scale

9. Lack of Repeatability

10. High Costs

11. Difficulty in Handling Complex Scenarios

12. Limited Automation Support